



Departamento de Lenguajes y  
Sistemas Informáticos

# Implementación de Servicios RESTful (III)

Integración

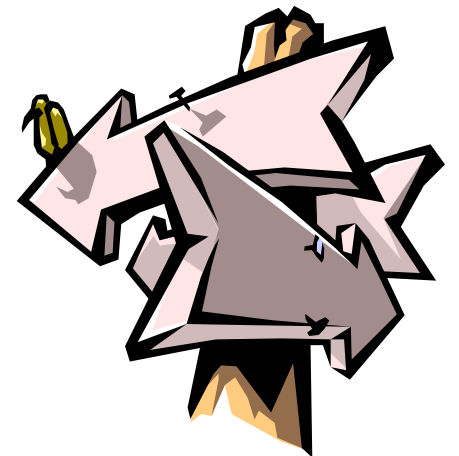


Arquitectura e Integración del Software

2025/2026

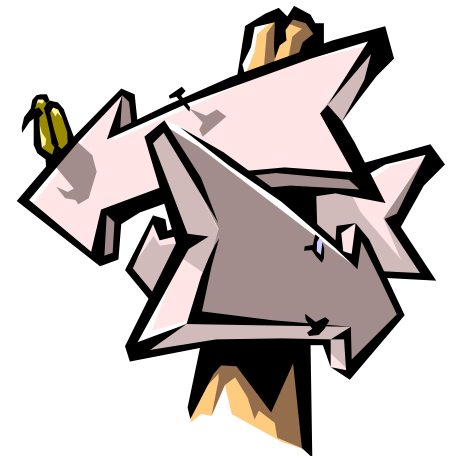
# Índice

- Documentación de APIs
- Aspectos avanzados
  - Paginación
  - Filtrado
  - Ordenación



# Índice

- **Documentación de APIs**
- Aspectos avanzados
  - Paginación
  - Filtrado
  - Ordenación



# Documentación de APIs

- La **calidad de la documentación** es un aspecto fundamental para el éxito de una API RESTful.
- Para la documentación de APIs vamos a utilizar el lenguaje de especificación **OpenAPI Specification (OAS)** y el editor online **Swagger**.
- **Swagger** es un conjunto de herramientas de software de código abierto que permite diseñar y construir APIs de acuerdo a los estándares de **especificación OpenAPI**, facilitando la generación de la **documentación** de las APIs.

# OpenAPI Specification

- La **especificación OpenAPI (OAS)** de una API se describe mediante un documento JSON o YAML indicando, entre otros:
  - Metadatos de la API (empresa, tipo de licencia, etc.)
  - Rutas y recursos.
  - Descripción detallada de operaciones y parámetros.
  - Códigos de estados usados por la API.

# OpenAPI Specification

```
openapi: 3.0.1
info:
  title: OpenAPI definition
  version: v0
servers:
- url: http://localhost:8080
  description: Generated server url
paths:
  /api/songs/{id}:
    put:
      tags:
      - song-controller
      operationId: update
      parameters:
      - name: id
        in: path
        required: true
        schema:
          type: integer
          format: int64
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Song'
            required: true
      responses:
        "204":
          description: No Content
```

# OpenAPI Specification

Para generar automáticamente los documentos de especificación de OpenAPI 3 para nuestra API, simplemente agregamos la siguiente dependencia a nuestro pom.xml:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-
ui</artifactId>
  <version>2.0.2</version>
</dependency>
```

# OpenAPI Specification

- Para acceder a la versión JSON de la documentación:

<http://localhost:8080/v3/api-docs/>

- Para acceder a la versión YAML de la documentación:

<http://localhost:8080/v3/api-docs.yaml>

- Además de generar la especificación OpenAPI 3 en sí, podemos integrar springdoc-openapi con la interfaz de usuario de Swagger para que podamos interactuar con nuestra especificación OpenAPI y practicar con los endpoints de nuestra API:

<http://localhost:8080/swagger-ui/index.html>

# OpenAPI Specification

Podemos añadir descripciones adicionales a la documentación de nuestra API usando anotaciones específicas de OpenAPI: (*@Operation*, *@ApiResponse*s, *@Parameter*, *@Tag*...)

## @Tag

```
@Tag(name = "Album", description = "Album management API")  
@RestController  
@RequestMapping("/api/albums")
```

## @Operation

```
@Operation(  
    summary = "Retrieve an Album by Id",  
    description = "Get an Album object by specifying its id",  
    tags = { "projects", "get" })
```

# OpenAPI Specification

Podemos añadir descripciones adicionales a la documentación de nuestra API usando anotaciones específicas de OpenAPI: (*@Operation*, *@ApiResponses*, *@Parameter*, *@Tag*...)

## @ApiResponses

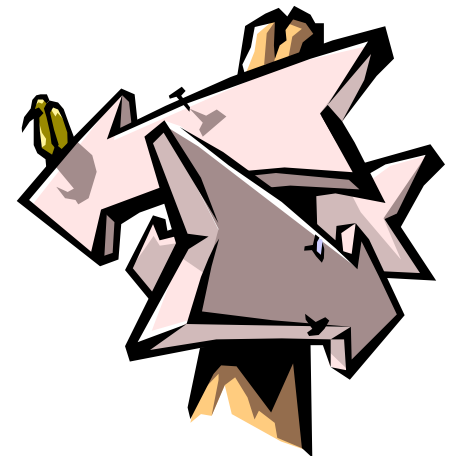
```
@ApiResponses({  
    @ApiResponse(responseCode = "200", description = "Listado de  
albums", content = { @Content(schema = @Schema(implementation = Album.class),  
mediaType = "application/json") }},  
    @ApiResponse(responseCode = "404", description="Album no  
encontrado", content = { @Content(schema = @Schema()) })
```

## @Parameter

```
public Album findOne(@Parameter(description = "id of album to be searched")  
@PathVariable Long id)
```

# Índice

- Documentación de APIs
- **Aspectos avanzados**
  - Paginación
  - Filtrado
  - Ordenación



# Aspectos avanzados

Podemos añadir algunas funcionalidades a nuestra API como paginación, filtrado y ordenación, utilizando la clase Pageable de JPA.

**Paginación:** Consiste en filtrar por el número resultados (size) y de página (page) los resultados obtenidos en una operación GET de nuestra API. Ej: <http://localhost:8080/api/albums?page=1&size=2>

```
public List<Album> findAll(@RequestParam(defaultValue = "0") int page,
                           @RequestParam(defaultValue = "10") int size)
{
    Page<Album> pageAlbums;
    Pageable paging = PageRequest.of(page, size);
    pageAlbums = albumRepository.findAll(paging);
    return pageAlbums.getContent();
}
```

# Aspectos avanzados

**Filtrado:** Consiste en filtrar los resultados obtenidos en una operación GET de nuestra API por el valor de alguno de sus atributos. Ej:  
<http://localhost:8080/api/albums?name=AlbumTitle>

```
Page<Project> findByName(String name, Pageable pageable);
```



```
public List<Album> findAll( @RequestParam(defaultValue = "0") int page,
                           @RequestParam(defaultValue = "10") int size,
                           @RequestParam(required = false) String name,
    ) {
    ...
    if (name != null)
        pageAlbums = albumRepository.findByName(name, paging);
    else
        pageAlbums = albumRepository.findAll(paging);
    return pageAlbums.getContent();
}
```



# Aspectos avanzados

**Ordenación:** Consiste en ordenar los resultados obtenidos en una operación GET de nuestra API por alguno de sus atributos. Ej:  
<http://localhost:8080/api/albums?order=name>

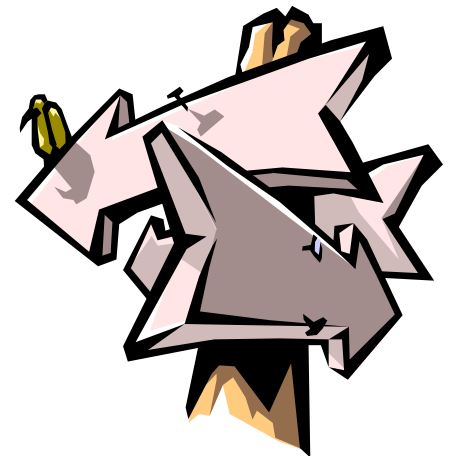
```
public List<Album> findAll( @RequestParam(defaultValue = "0") int page,
                           @RequestParam(defaultValue = "10") int size,
                           @RequestParam(required = false) String name,
                           @RequestParam(required = false) String order) {
    Pageable paging;

    if(order != null) {
        if (order.startsWith("-"))
            paging = PageRequest.of(page, size, Sort.by(order.substring(1)).descending());
        else
            paging = PageRequest.of(page, size, Sort.by(order).ascending());
    }
    else
        paging = PageRequest.of(page, size);
    ...
}
```



# Índice

- Documentación de APIs
- Aspectos avanzados
  - Paginación
  - Filtrado
  - Ordenación
- **Referencias**



# Referencias

- Documentación de APIs:

- <https://openapis.org/>
- <https://www.bezkoder.com/spring-boot-swagger-3/>

- Aspectos avanzados:

- <https://www.bezkoder.com/spring-boot-pagination-filter-jpa-pageable>
- <https://www.baeldung.com/spring-data-jpa-pagination-sorting>
- <https://www.bezkoder.com/spring-boot-jpa-h2-example>

# Disclaimer and Terms of Use

All material displayed on this presentation is for teaching and personal use only.

Many of the images that have been used in the presentation are Royalty Free images taken from <http://www.everystockphoto.com/>. Other images have been sourced directly from the Public domain, from where in most cases it is unclear whether copyright has been explicitly claimed. Our intention is not to infringe any artist's copyright, whether written or visual. We do not claim ownership of any image that has been freely obtained from the public domain. In the event that we have freely obtained an image or quotation that has been placed in the public domain and in doing so have inadvertently used a copyrighted image without the copyright holder's express permission we ask that the copyright holder writes to us directly, upon which we will contact the copyright holder to request full written permission to use the quote or images.